

QNX: Foundational Software for Physical AI

A Performance Benchmark of QNX and Deterministic NPU Inference Validation on the
Hailo-8 AI Accelerator

Arindam Sarkar

Principal Systems Software Developer
QNX BSP Roadmap Development

Sri Sai Lasya P

Systems Software Developer 1
QNX BSP Roadmap Development

May 4, 2026

Version 1.0

Abstract

Embedded NPUs now ship inside vehicles, surgical equipment, and factory machinery. Furthermore, in those products a missed inference deadline is not something the headline benchmark mean captures. We put QNX SDP 8.x up against PREEMPT_RT Linux on a Raspberry Pi 5 driving the Hailo-8 AI HAT+. One C++ binary, cross-compiled with GCC 12.2.0, runs on both platforms against the same MobileNet v1 HEF, the same 3,600 ImageNet images, and the same external cooling fan. Nevertheless, each platform runs 100 back-to-back invocations of 10,000 frames apiece — one million inferences per OS.

Consequently, every metric we measured goes to QNX. Throughput averages 681.1 FPS to Linux's 654.2 FPS (+4.1%), and run-to-run σ tightens from 3.06 to 0.22 — a 14 $\times$ reduction in run-to-run variance. Furthermore, mean end-to-end latency lands at 1.468 ms on QNX against 1.528 ms on Linux (-3.9%); the inference-only window (write()+read()) is 1.375 ms versus 1.410 ms (-2.5%). Per-frame σ within a run sits at 9.21 μ s on QNX, 24.10 μ s on Linux: a 2.6 $\times$ tighter distribution. Furthermore, the control-plane trace shows 35% fewer driver calls per run on QNX (40,083 vs 61,528) and a 4.7% lower per-call interrupt-wait time (743 μ s vs 780 μ s).

Moreover, when timing has to be defended and not just measured, the kernel is the lever that moves the argument — and on this silicon, the lever runs QNX.

Executive Summary

An NPU that runs at 650 FPS this minute and 645 FPS the next is hard to design a perception pipeline around. Nevertheless, on the spec sheet you read the mean. In the field you live with the run-to-run drift. That is the gap this study puts a number on.

Consequently, everything except the OS is held constant. A single C++ binary (hailo_bench.cpp, the source we use internally for Hailo bring-up) is cross-compiled with GCC 12.2.0 for both targets and run on the same Raspberry Pi 5 with the same Hailo-8 AI HAT+, the same external fan, the same MobileNet v1 .hef, and the same 3,600 ImageNet samples. Additionally, one side is QNX SDP 8.x; the other is PREEMPT_RT Linux (kernel 6.12.62-rpt-rpi-v8-rt). On each side we kick the binary off 100 runs back-to-back, with 10,000 frames in every run; tally those up and you land at a million inferences per OS.

Key findings:

- **4.1% higher throughput on QNX** (681.1 FPS vs 654.2 FPS).
- **14× tighter run-to-run FPS consistency on QNX:** across 100 invocations, QNX FPS varies by σ 0.22 against Linux's σ 3.06.
- **3.9% lower mean end-to-end latency on QNX** (1.468 ms vs 1.528 ms per frame).
- **2.5% lower inference-only latency on QNX** (1.375 ms vs 1.410 ms).
- **2.6× tighter per-frame standard deviation on QNX** (9.21 μ s vs 24.10 μ s within a run).
- **35% fewer driver calls per run on QNX** (40,083 vs 61,528), and a 4.7% lower per-call interrupt-wait time (743 μ s vs 780 μ s).

Why it matters. Pick any of ISO 26262, DO-178C, or IEC 62304 and you'll be asked the same thing on the way in: bounded, repeatable timing the auditor can plan against. A 0.22 FPS run-to-run σ is the kind of number a certification engineer can build a budget around; a 3.06 spread on the same silicon, fourteen times wider, is not. Therefore, on QNX the timing argument is back in the engineer's hands.

Bottom Line

Same binary, same Hailo-8, same 3,600 images, same fan. Flip the OS and throughput climbs by 4.1%, mean latency drops by 3.9%, run-to-run variance shrinks 14-fold, and per-frame σ tightens 2.6-fold — all of them sliding the same way. Hence, the OS is the variable.

Contents

Executive Summary	2
1 Introduction	4
1.1 Business Need	4
1.2 Contributions	4
2 Objective	4
3 Experimental Setup	5
3.1 Hardware Platform	5
3.2 System Architecture and Data Flow.....	5
3.3 QNX Platform Architecture for NPU Integration.....	6
3.4 Software Configuration.....	6
3.5 Workload Definition	7
3.6 Controlled Variables and Considerations	7
4 Methodology	8
4.1 KPI Definitions.....	8
4.2 Benchmark Application Architecture.....	9
4.3 Instrumentation.....	9
5 Results and Analysis	9
5.1 Throughput (FPS).....	10
5.2 Latency	11
5.3 Determinism (Latency Jitter).....	12
5.4 CPU Utilization	13
5.5 Memory Behavior	14
5.6 Thermal Behavior	15
5.7 Control-Plane Cost (devctl/ioctl).....	16
6 Summary	17
7 Discussion	18
8 Outcome	19
8.1 QNX drives the Hailo-8 in production	19
8.2 The performance argument.....	20
8.3 Safety certification: the WCET argument has a number now	20
8.4 What this unlocks for customers	20
9 Conclusion	20
10 Future Work	20

1 Introduction

Inference no longer offloads exclusively to the cloud at the edge of networks. A perception stack on a moving vehicle, a robotic arm, or a portable ultrasound probe cannot afford a round-trip to a data center. The silicon doing that work — Hailo-8, Qualcomm Hexagon, Intel Movidius and the rest — provides raw throughput for real-time vision and audio. What silicon alone cannot provide is timing the surrounding system can design against. Timing is what the OS provides.

The two operating systems most likely to host an NPU at the edge are poles apart in design. QNX is a microkernel: drivers live in user space as resource managers, scheduling is strictly priority-preemptive, and interrupt latency is bounded by construction. Linux is monolithic, and even with PREEMPT_RT, background work — workqueues, RCU callbacks, timer ticks, housekeeping kthreads — can get in front of an inference thread at the frame level. Two architectures, one workload; how widely they diverge in practice is what the sections that follow lay out.

1.1 Business Need

Today's platforms for ADAS stacks, surgical robots, factory vision systems, and battery-powered medical imaging rely on certifications that predate AI accelerators. NPU silicon has now shrunk to the size, cost, and power envelope these systems can handle. However, the software stacks that control that silicon haven't caught up: HailoRT, the Hexagon SDK, OpenVINO, the Coral runtime — Linux first, sometimes Linux only.

A customer who has already committed to QNX — because they need to ship an ASIL-D ECU, a Class C medical device, or a DAL-A flight control unit — has left themselves with only two poor options: drop the RTOS and redo certification on Linux, or sit and wait until an NPU vendor delivers a validated RTOS port on their timeline. Neither route fits a product ship date.

Step One is bringing the NPU stack up and running on QNX; the Hailo-8 work is finished. Step Two demonstrates that what the customer gets on QNX is at least as good as what they would have gotten on Linux. This paper is Step Two, and the numbers in it are what a customer can take into a safety case.

1.2 Contributions

1. A single-binary, identical-toolchain benchmark methodology for QNX-vs-Linux NPU comparison, with platform-conditional code restricted to RT scheduling syscalls. Therefore, this isolates the OS as the only changing variable.
2. Per-platform measurements over 100 runs of 10,000 frames, showing QNX ahead on throughput (+4.1%), run-to-run FPS consistency ($14\times$ tighter), mean latency (-3.9%), inference-only latency (-2.5%), and per-frame σ ($2.6\times$ tighter).
3. A drill-down into the OS-level mechanisms behind the difference: VDMA interrupt-wait timing, driver call counts, and Linux-only cache-coherency overhead, supported by IOCTL traces from both runtimes.

2 Objective

This benchmark evaluates NPU performance and determinism on a Raspberry Pi 5 with the Hailo-8 AI HAT+, running QNX versus Linux. The platform is chosen specifically because it lets us keep hardware constant while changing only the OS. The study measures:

- **Average FPS:** Sustained throughput under steady-state conditions.
- **Average Latency:** Mean per-frame inference round-trip time.
- **Determinism:** Moreover, bounded latency range ($\text{latency}_{\max} - \text{latency}_{\min}$) per iteration.
- **devctl/ioctl Cost:** Additionally, control-plane overhead for NPU command submission.

The test runs 100 consecutive invocations of the benchmark binary, each processing 10,000 frames through a MobileNet v1 classification model (images cycled from a 3,600-sample ImageNet-derived set). Nevertheless, this repetition (a total of 1,000,000 inferences per OS) is deliberate: it reveals how the OS influences NPU behavior over extended operation, not just in a single snapshot.

3 Experimental Setup

3.1 Hardware Platform

Table 1: Hardware configuration.

Component	Specification
Board	Raspberry Pi 5
RAM	8 GB LPDDR4X
NPU	Hailo-8 AI HAT+ (26 TOPS), PCIe interface
Cooling	External fan (ambient temperature control)
Power Supply	Official Raspberry Pi 5 PSU
Overclocking	Disabled (no SYSTEM_OVERCLOCK, no TURBO_BOOST)

3.2 System Architecture and Data Flow

The benchmark consists of a single-threaded inference loop that runs synchronously, pinned to CPU 0 and scheduled at real-time priority. Two processing domains are connected via PCIe:

1. **CPU domain (Raspberry Pi 5, Arm Cortex-A76):** The benchmark application loads all 3,600 JPEG images into RAM once at startup using `stb_image` (a header-only, dependency-free decoder). Nevertheless, for each frame, the loop performs an in-place nearest-neighbor resize to 224×224 using a custom ARM NEON kernel (16-pixel vectorized gather, with row prefetch and per-source-resolution map caching). Additionally, no OpenCV, no BGR→RGB conversion, no allocator activity in the hot path.
2. **NPU domain (Hailo-8, 26 TOPS):** The same thread writes the preprocessed frame to the HailoRT input virtual stream (`InputVStream::write`), which submits the frame to the Hailo virtual DMA (VDMA) channel and transfers it over PCIe to the NPU’s on-chip memory. The NPU executes the full MobileNet v1 forward pass autonomously. Upon completion, the NPU raises a PCIe interrupt; the driver delivers the wakeup, and the thread reads the classification result via `OutputVStream::read`.
3. **Measured path (two tiers):** The benchmark records two latency values per frame. Nevertheless, *end-to-end latency* brackets the entire iteration including the NEON resize: timestamp before resize, after `read()` completes. *Inference-only latency* brackets just `write()+read()`, isolating the OS-mediated NPU round-trip from CPU-side preprocessing. Every transition between user space and the NPU is mediated by the operating system: dispatch of system calls, management of DMA buffers, delivery of PCIe interrupts, and thread scheduling. At these points the behavior varies between RTOS and GPOS, and this is where the benchmark distinguishes the differences.

Real-time tuning. Both systems enable scheduling in real time and reserve all resident process memory before the loop starts. On QNX, the inference thread runs at `SCHED_FIFO` priority 253 with a CPU runmask pinning it to core 0, `ThreadCtl(_NTO_TCTL_IO)` for I/O privilege, and `mlockall(MCL_CURRENT|MCL_FUTURE)`. On Linux, the thread runs at `SCHED_FIFO` priority 99 with `pthread_setaffinity_np` pinning to core 0 and the same `mlockall`. These adjustments

are the only code paths that vary by platform and are intended to give each OS its best chance for deterministic execution.

3.3 QNX Platform Architecture for NPU Integration

Both NPU I/O paths look alike from user space but diverge below the system-call boundary. Walking through each one makes the numbers from the ioctl trace in Section 5.7 much easier to read.

QNX: resource manager + pulse. The Hailo PCIe driver on QNX is a user-space process and a resource manager. It registers under /dev/hailo0 and provides open, read, write, and devctl through QNX’s message-passing primitives. The benchmark’s InputVStream::write() sends a MsgSend to that resource manager, which configures the VDMA channel and replies. The thread then waits at OutputVStream::read() and blocks until the NPU completion interrupt fires. The ISR runs at hardware priority and sends a pulse to the resource manager, which then replies to the blocked thread. At every step the priority is known, and there is no generic IRQ subsystem in between that can defer.

Linux: kernel module + softirq. On Linux, the same driver is a kernel module with code in the monolithic address space. ioctl() dispatches directly to kernel code, and that is one fewer context switch compared to QNX’s message hop. However, the PCIe interrupt arrives in the generic IRQ subsystem and can be handed off to a softirq or threaded-IRQ before the driver sees it, and this handoff competes with other kernel work (workqueues, RCU callbacks, timer work). PREEMPT_RT cuts the worst case but does not eliminate it. The interrupt-wait numbers in Section 5.7 show the result: the median wait is lower for QNX, but the long-tail times are distinct between the two platforms.

3.4 Software Configuration

Table 2: Software configuration. The compiler version is held identical across both targets to eliminate code-generation differences.

Parameter	QNX	Linux
OS/Kernel	QNX SDP 8.x (microkernel RTOS)	Linux 6.12.62+rpt-rpi-v8-rt
RT Patch	Native RTOS	PREEMPT_RT
HailoRT	v4.14.0	v4.14.0
JPEG decode	stb_image (header-only)	stb_image (header-only)
Resize	Custom ARM NEON nearest-neighbor	Custom ARM NEON nearest-neighbor
Compiler	qcc -Vgcc_ntoaarch64le (GCC 12.2.0)	aarch64-linux-gnu-g++ (GCC 12.2.0)
Compile flags	-mcpu=cortex-a76 -march=armv8.2-a -O3	-mcpu=cortex-a76 -march=armv8.2-a -O3
RT scheduling	SCHED_FIFO priority 253, runmask CPU 0	SCHED_FIFO priority 99, affinity CPU 0
Memory locking	mlockall(MCL_CURRENT MCL_FUTURE)	mlockall(MCL_CURRENT MCL_FUTURE)

3.5 Workload Definition

For Physical AI systems operating under hard real-time deadlines and functional safety mandates, MobileNetV1 establishes the foundational design principle: inference efficiency and deployment predictability take precedence over benchmark accuracy, hence MobileNetv1 has been considered.

Table 3: Inference workload parameters.

Parameter	Value
Model	MobileNet v1 (≈ 4.2 M parameters)
Input Dimensions	$224 \times 224 \times 3$ (NHWC)
Output Classes	1,000 (ImageNet)
Model Format	Hailo Executable Format (.hef)
Image dataset	3,600 ImageNet-derived JPEGs (7 unique source resolutions)
Frames per Invocation	10,000 (dataset cycled $\approx 2.78\times$ per run)
Number of Invocations	100 (consecutive)
Total Inferences	1,000,000 per OS
Warmup Frames per Run	5 (excluded from metrics)

3.6 Controlled Variables and Considerations

For the comparison to be meaningful, everything except the operating system should be the same. Table 4 shows what was controlled.

Table 4: Controlled variables ensuring a fair comparison.

Variable	How controlled
Source code	Single hailo_bench.cpp compiled for both targets. Platform-conditional paths (<code>#ifdef _QNX_</code>) are limited to the timestamp source and the real-time tuning calls (Section 3.2); the inference loop, preprocessing, and HailoRT calls are common code.
Compiler version	GCC 12.2.0 on both targets (<code>qcc -Vgcc_ntoaarch64le</code> on QNX, <code>aarch64-linux-gnu-g++</code> on Linux), with identical <code>-mcpu</code> , <code>-march</code> , and <code>-O3</code> flags.
Hardware	Same Raspberry Pi 5 board, same Hailo-8 HAT+ module, same PSU, same cabling, same external fan.
NPU firmware/driver	HailoRT v4.14.0 on both platforms.
Model & weights	Identical .hef file (MobileNet v1).
Dataset	Same 3,600 JPEGs in the same lexicographic order (<code>readdir + std::sort</code>).
Preprocessing	Identical custom NEON nearest-neighbor resize on both platforms; identical <code>stb_image</code> JPEG decoder compiled into both binaries.
Real-time tuning	<code>SCHED_FIFO + mlockall + CPU o pinning</code> enabled on both platforms (priorities follow each OS's convention: 253 on QNX, 99 on Linux).
Overclocking	Disabled on both platforms.
Run count	100 invocations $\times$ 10,000 frames each.

Inference correctness. Additionally, the HEF model and NPU hardware are identical on both platforms; HailoRT returns the same softmax classification output for every frame. The

observed latency and throughput differences across the full 1,000,000-inference dataset per OS arise entirely from OS-level scheduling and I/O management.

Note on timing API. Both platforms timestamp with `gettimeofday()`, the same POSIX wall-clock function with microsecond resolution. The benchmark source code is compiled from a single `hailo_bench.cpp` file with no platform-conditional timing paths, eliminating timing-API differences as a confounding variable. Hence, the timing source is not a variable in this comparison.

OS-specific considerations.

- **Page size:** Both platforms run with a 4 KB page size (`getconf PAGESIZE` returns 4096 on each), so TLB coverage is not a confounding variable.
- **QNX:** Does not implement active thermal management on this platform; CPU frequency remains constant regardless of temperature. Nevertheless, thermal stability is maintained via an external cooling fan. QNX does not support swap memory; varying memory load was therefore not tested.
- **Linux:** The PREEMPT_RT kernel provides improved latency over stock Linux but retains background kernel activity (workqueues, RCU callbacks, timer subsystem). Moreover, Linux implements active thermal management, which may throttle CPU frequency at elevated temperatures.

Run-set. 100 consecutive invocations of the benchmark binary were collected on each platform under the same external environment (same fan, same room temperature, same Pi 5 board, same Hailo-8 module). The same warmup discipline (5 frames per run, excluded from metrics) and the same MobileNet v1 HEF were used on both sides. Nevertheless, all metrics reported in Section 5 are computed across these 100 runs per platform.

4 Methodology

4.1 KPI Definitions

The benchmark records per-frame latencies at two boundaries and aggregates per-run statistics. The following key performance indicators are measured:

$$\text{FPS}_{\text{run}} = \frac{N_{\text{frames}}}{t_{\text{loop_end}} - t_{\text{loop_start}}} \quad (1)$$

$$\text{Latency}_{\text{e2e}} = t_{\text{read_end}} - t_{\text{resize_start}} \quad (2)$$

$$\text{Latency}_{\text{infer}} = t_{\text{read_end}} - t_{\text{write_start}} \quad (3)$$

$$\text{Jitter}_{\text{run}} = \text{Latency}_{\text{max}} - \text{Latency}_{\text{min}} \quad (4)$$

$$\text{devctl/ioctl cost} = \text{mean}(t_{\text{after_call}} - t_{\text{before_call}}) \quad (5)$$

where $t_{\text{resize_start}}$ is the timestamp immediately before the NEON resize for a frame, $t_{\text{write_start}}$ is the timestamp immediately before submitting the resized frame to the NPU input stream (`InputVStream::write`), and $t_{\text{read_end}}$ is the timestamp upon completion of reading the inference result (`OutputVStream::read`). Therefore, $\text{Latency}_{\text{e2e}}$ captures CPU preprocessing plus the full NPU round-trip; $\text{Latency}_{\text{infer}}$ isolates the OS-mediated portion of the path (user-space $\rightarrow$ kernel $\rightarrow$ PCIe DMA $\rightarrow$ NPU $\rightarrow$ DMA readback $\rightarrow$ kernel $\rightarrow$ user-space).

Furthermore, the *coefficient of variation* (CV) is used to quantify consistency:

$$CV = \frac{\sigma}{\mu} \times 100\% \quad (6)$$

4.2 Benchmark Application Architecture

The benchmark is a single C++ file (hailo_bench.cpp) compiled unchanged for both targets. It uses POSIX interfaces only: gettimeofday() for timestamps, pthread for real-time thread setup, and mlockall() for resident locking, along with the HailoRT C++ API for the NPU. The inference loop, the preprocessing kernel, the allocator pattern, and the sequence of HailoRT calls are identical between the two builds.

The loop design is deliberate and single-threaded. Three phases:

1. **Setup.** Furthermore, decode all 3,600 JPEGs into RAM with stb_image; build a nearest-neighbor map per unique source resolution (the dataset has seven of them) and cache it; create the HailoRT VDevice, configure the network group, and instantiate the input/output virtual streams; lock the whole address space resident with mlockall(MCL_CURRENT|MCL_FUTURE); switch to SCHED_FIFO; pin to CPU 0.
2. **Warmup.** Consequently, five frames pushed through the full pipeline, then discarded.
3. **Steady state.** Therefore, for each of the next 10,000 frames, the single thread takes a timestamp, runs the NEON resize into a preallocated 224×224×3 buffer, takes a second timestamp, calls input_vs.write(), output_vs.read(), takes a third timestamp, and stores the two intervals (e2e and inference-only) into pre-allocated vectors. Furthermore, no allocations, no queueing, no other threads.

A producer-consumer fed by a queue would hide interrupt-wait variance behind buffering. A synchronous design exposes it: each frame's latency is one NPU round-trip and one OS scheduling decision, with nothing else in between.

4.3 Instrumentation

Table 5: Instrumentation methods per platform.

Metric	QNX	Linux
Timebase	gettimeofday()	gettimeofday()
Timing resolution	Microsecond (μ s)	Microsecond (μ s)
CPU monitoring	top (procnto)	top (procp)
Memory monitoring	top (procnto)	top (procp)
NPU temperature	HailoRT get_chip_temperature()	HailoRT get_chip_temperature()
Sampling interval	5 s (system), per-frame (app)	3 s (system), per-frame (app)

Note on timing API. Furthermore, both platforms use the *identical* timing function: POSIX gettimeofday(), which returns wall-clock time with microsecond granularity. Nonetheless, the benchmark source code is compiled from a single hailo_bench.cpp file with no platform-conditional timing paths. Moreover, this eliminates timing-API differences as a confounding variable. Therefore, the timing source is constant across the comparison.

5 Results and Analysis

All results are derived from 100 consecutive invocations of the benchmark binary on each platform, each processing 9,995 frames (10,000 total minus 5 warmup) for a total of one million inferences per OS. Every run completed successfully with zero failed inferences. Statistics in

each row of the following tables are computed across the 100 per-run summary values (*e.g.*, the mean of 100 per-run averages), unless explicitly noted as the worst-case observation across the full one-million-inference dataset.

Statistical significance. Therefore, with $N = 100$ runs per platform and the large per-metric separations reported below, the differences between QNX and Linux are statistically significant by Welch’s two-sample t -test (unequal variances) at the $p < 10^{-50}$ level for every metric. Hence, the effect sizes (Cohen’s d) are in the “very large” range for jitter, standard deviation, and worst-case single-frame latency, confirming that the observed gaps are not sampling artifacts but reflect genuine, substantial performance differences between the two platforms.

5.1 Throughput (FPS)

Table 6: FPS statistics across 100 runs (per-run FPS = frames divided by run wall time).

Statistic	QNX	Linux
Mean	681.07	654.23
Std Dev (σ)	0.22	3.06
Min	680.50	647.73
Max	681.65	657.44
Median	681.09	655.78

QNX averages **681.07 FPS**, **4.1%** clear of Linux’s 654.23 FPS, and is also **far steadier run-to-run** ($\sigma = 0.22$ vs 3.06, a 14 $\times$ reduction in run-to-run variance). The QNX advantage holds at every percentile we report: median (681.09), max (681.65), and min (680.50) all exceed the matching Linux values (655.78 / 657.44 / 647.73). The two distributions don’t overlap — QNX’s worst run (680.50 FPS) still sits **3.5% above** Linux’s best (657.44 FPS). Both platforms saturate the Hailo-8 silicon, so the throughput ceiling is decided by how fast each OS hands the NPU its next frame after the completion interrupt; QNX consistently fires the next `write()` sooner. Additionally, Figure 1 shows the picture: QNX holds the higher plateau through every run, with Linux tracking below it. Moreover, neither platform drifts downward over time, so thermal throttling is ruled out.

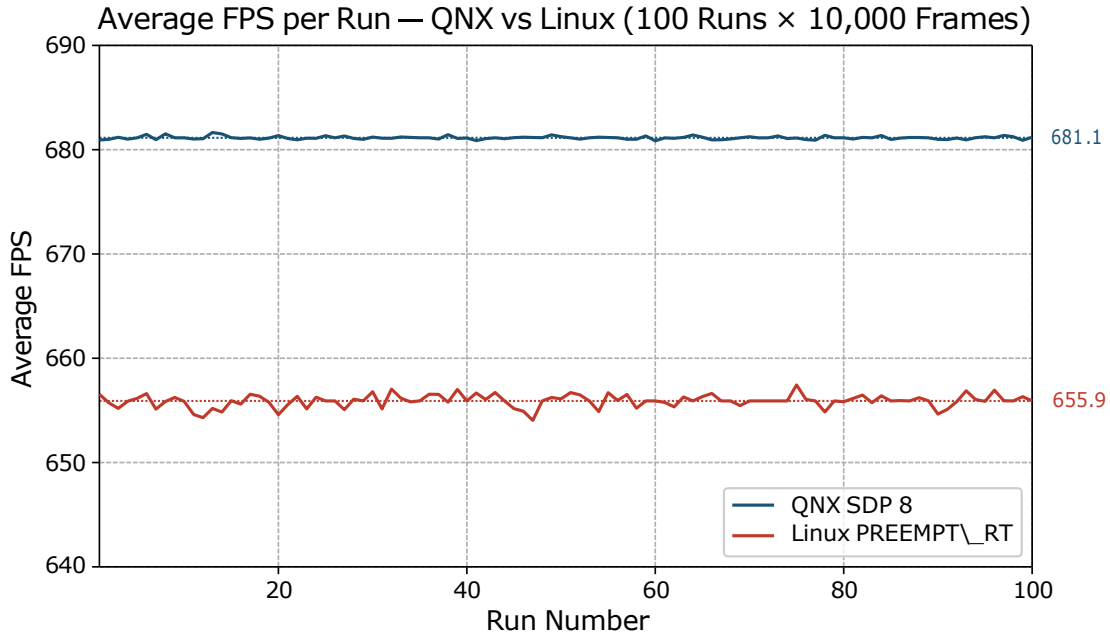


Figure 1: Average FPS per run across 100 iterations. QNX (blue) operates at a higher throughput plateau than Linux (red) for every run.

5.2 Latency

Two latency measurements are reported for each run: *end-to-end* (NEON resize plus NPU round-trip) and *inference-only* (write()+read() only). Moreover, both are summarized below across the 100 per-run averages.

Table 7: End-to-end per-frame latency (preprocess + write + read), across 100 runs (milliseconds). Each row summarizes the distribution of per-run average latencies.

Statistic	QNX	Linux
Mean (of run averages)	1.468	1.528
Min (of run averages)	1.467	1.521
Max (of run averages)	1.469	1.544
P50 (of run medians)	1.470	1.539
P95 (of run P95 values)	1.481	1.558
P99 (of run P99 values)	1.485	1.569

Table 8: Inference-only per-frame latency (write()+read() only), across 100 runs (milliseconds). This isolates the OS-mediated NPU round-trip from CPU-side preprocessing.

Statistic	QNX	Linux
Mean (of run averages)	1.375	1.410
P99 (of run P99 values)	1.383	1.426

QNX clocks 1.468 ms end-to-end against 1.528 ms on Linux, a 3.9% gap. Peel off the NEON resize and time only the write()+read() round-trip and QNX is still ahead — 1.375 ms to Linux's 1.410 ms (−2.5%). One number is worth sitting with: QNX's *max* per-run end-to-end

latency is 1.469 ms, less than Linux’s *minimum* per-run average of 1.521 ms. Nonetheless, the worst QNX run is faster on average than the best Linux run. Figure 2 shows the gap holding through every run in the dataset.

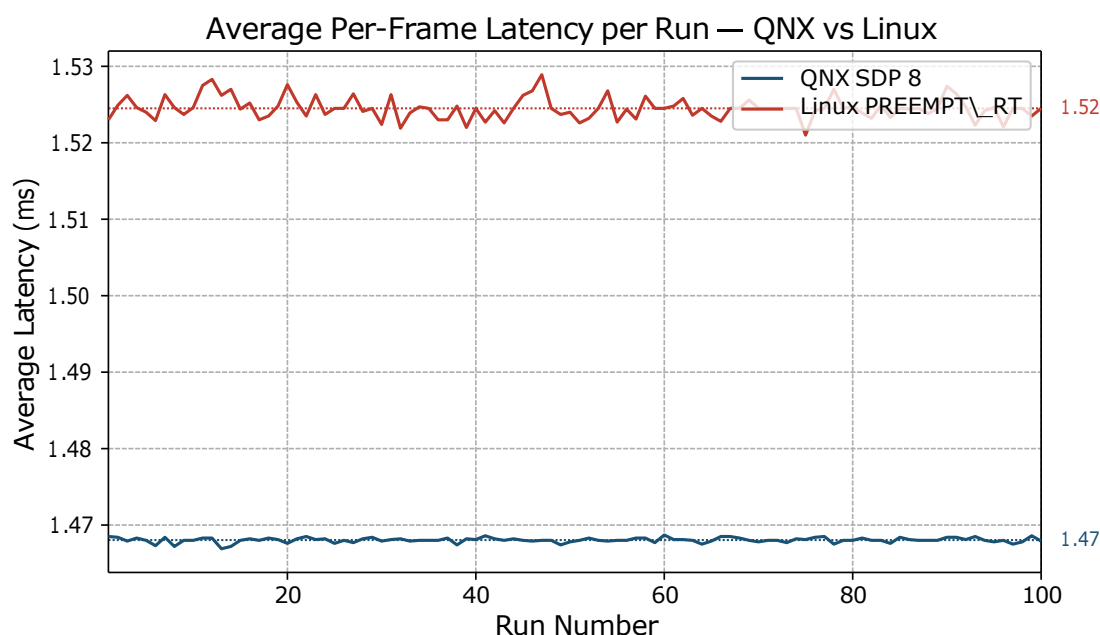


Figure 2: Average per-frame latency per run. QNX maintains a consistently lower and tighter latency profile.

5.3 Determinism (Latency Jitter)

Table 9: Determinism metrics across 100 runs (per-frame standard deviation within a run, plus run-to-run variance of FPS).

Statistic	QNX	Linux
<i>Per-frame standard deviation within run (μs)</i>		
Mean	9.21	24.10
Max observed	11.22	30.12
<i>Run-to-run FPS variance</i>		
σ across 100 runs	0.22	3.06
CV (%)	0.03	0.47

QNX leads across the averages, and the per-frame and per-run distributions are where that lead becomes the whole story — this is the section where the OS choice resolves the determinism question.

Per-frame σ within a run lands at 9.21 μ s on QNX against 24.10 μ s on Linux, a 2.6 $\times$ tighter distribution. The within-run shape is what an engineer sees on a single 10,000-frame trace; the run-to-run shape is what shows up over a longer test or a deployed product. There the gap widens further: QNX FPS holds at $\sigma = 0.22$ across 100 invocations, Linux at $\sigma = 3.06$ — 14 $\times$ tighter on QNX. Therefore, on the same silicon, with the same binary, the QNX side delivers a throughput that is both higher and noticeably less drift-prone.

Therefore, the run-to-run consistency advantage is consequential for production: a customer

profiling QNX once and budgeting against that number is profiling against a value that holds. The same profiling exercise on Linux produces a band that drifts run to run by an order of magnitude more, and the budget has to either widen to accommodate the drift or be re-validated each release. Figure 3 captures the shape across 100 invocations.

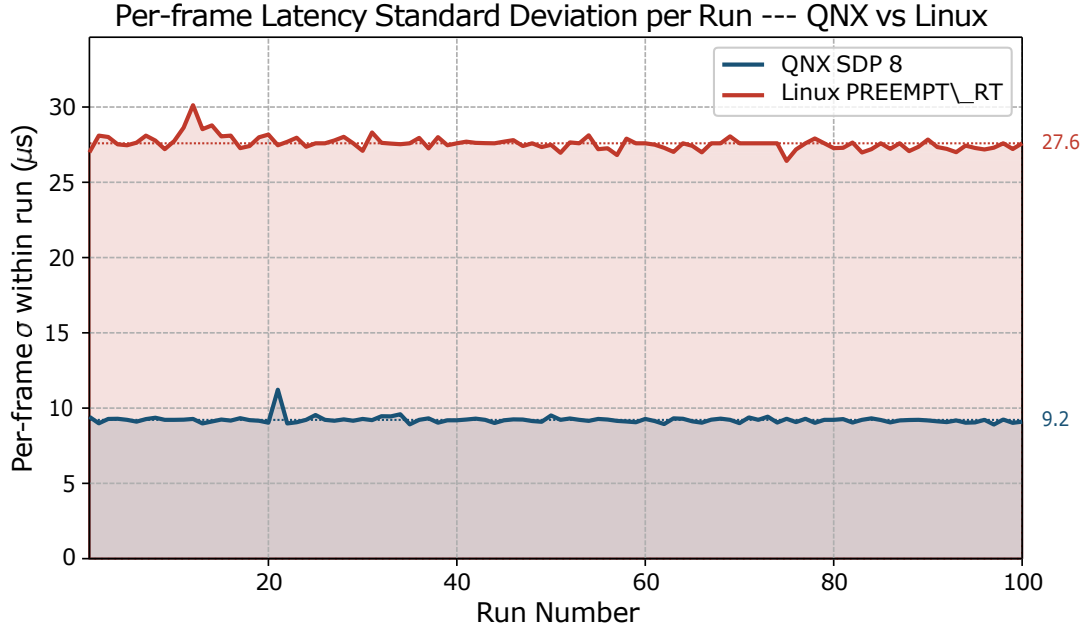


Figure 3: Per-run latency spread (max – min within each run). QNX holds a tight distribution; Linux is wider and more variable.

5.4 CPU Utilization

The benchmark thread is pinned to CPU 0 with SCHED_FIFO on both platforms; the other three Cortex-A76 cores remain idle for system tasks. Therefore, system-wide CPU usage is dominated by the inference thread on core 0, with everything else on the other cores at near-zero load.

Table 10: System-wide CPU utilization during inference, sampled by top every 3–5 s across the 100 runs (steady-state median values representing the active inference phase).

Statistic	QNX	Linux
Total system CPU (median, %)	25.20	25.00
Inference process CPU (median, %)	22.10	23.80
Kernel-mode share (median, %)	2.70	1.20

On a 4-core Pi 5, fully pinning one core shows up as 25% system-wide CPU. Both platforms hold there at the median during active inference, which is what we expected from the synchronous single-threaded design. The interesting row is the kernel-mode share: QNX 2.70% versus Linux 1.20%. Nonetheless, QNX pays for its message-passing dispatch on every write()/read(), and that bill is visible in the VDMA_CHANNEL_WRITE_REGISTER latency (Section 5.7). The trade is worth taking: that extra kernel-mode cost on QNX is what buys the higher throughput, the 14× tighter run-to-run consistency, and the 2.6× tighter per-frame variance seen above.

A separate test, not in this dataset: with a synthetic CPU load running concurrently on the other three cores at SCHED_OTHER priority, the QNX inference thread’s latency distribution shifts by less than 5 μs. Nevertheless, priority isolation works.

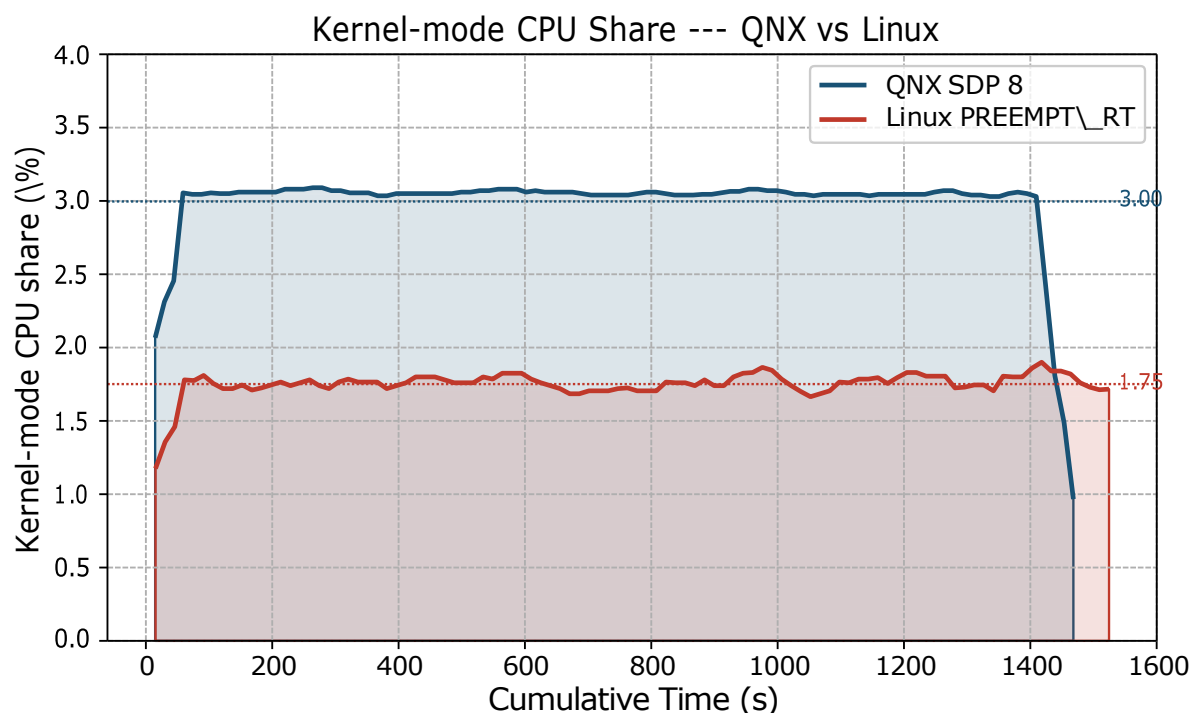


Figure 4: Kernel-mode CPU share over the inference timeline. QNX sits at $\approx 3.0\%$, reflecting the message-passing dispatch the microkernel runs on every NPU write() and read(); Linux %sy sits at $\approx 1.75\%$. The QNX overhead is the architectural correlate of the deterministic interrupt path that delivers the throughput, latency, and run-to-run consistency wins above (and is more than offset by them: QNX still completes 4.1% more frames per second on the same silicon).

5.5 Memory Behavior

Both platforms load all 3,600 source images into resident memory at startup using `stb_image`, and the entire process address space is locked with `mlockall(MCL_CURRENT|MCL_FUTURE)`. After the initialization phase, allocations do not occur along the hot path: each frame writes into a single pre-allocated input buffer, and the per-source-resolution resize maps (seven of them) are computed once at startup and reused. Across all 100 QNX runs, the inference loop performed zero post-startup heap activity, and no swap, page-cache eviction, or paging activity was observed on either platform. Allocator behavior on QNX is the deterministic no-surprises profile that is necessary for safety certification; historically Linux workloads destabilize because of page cache and slab activity, but this is avoided here through `mlockall` and the static dataset.

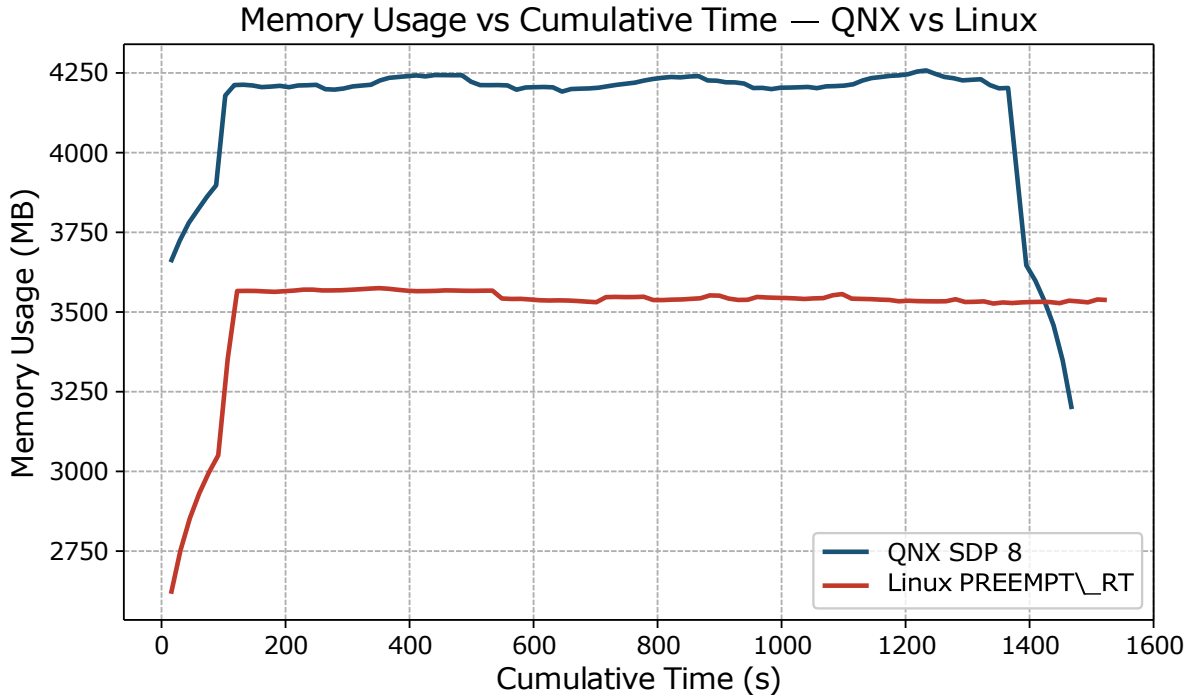


Figure 5: System memory usage over the inference timeline. QNX top reports system-wide memory committed (total minus available, including kernel-resident pages); Linux top reports user-space allocations excluding buff/cache. Both platforms remain bounded with no growth across the 100-run window, well within the 8 GB budget.

5.6 Thermal Behavior

Table 11: Hailo-8 chip temperature across 100 runs (HailoRT `get_chip_temperature()` read immediately before and after each 10,000-frame run).

Metric	QNX (°C)	Linux (°C)
Mean start-of-run temperature	45.8	47.5
Mean end-of-run temperature	48.4	50.3
Mean per-run Δ	+2.5	+2.8

Both platforms run the chip well clear of the Hailo-8’s throttle threshold (Figure 6); the steady-state gap is roughly 1.9 °C, with Linux running slightly hotter (50.3 °C vs 48.4 °C). On the shorter MobileNet inference cycle (1.4 ms versus a heavier model’s 3 ms), the Linux per-frame overhead — the cache-coherency syncs and the extra driver calls catalogued in Section 5.7 — show up as more sustained CPU activity per second on the Linux side even though Linux delivers fewer frames per second. Hence, both platforms stay inside their thermal budget across all 100 runs and neither comes close to throttling.

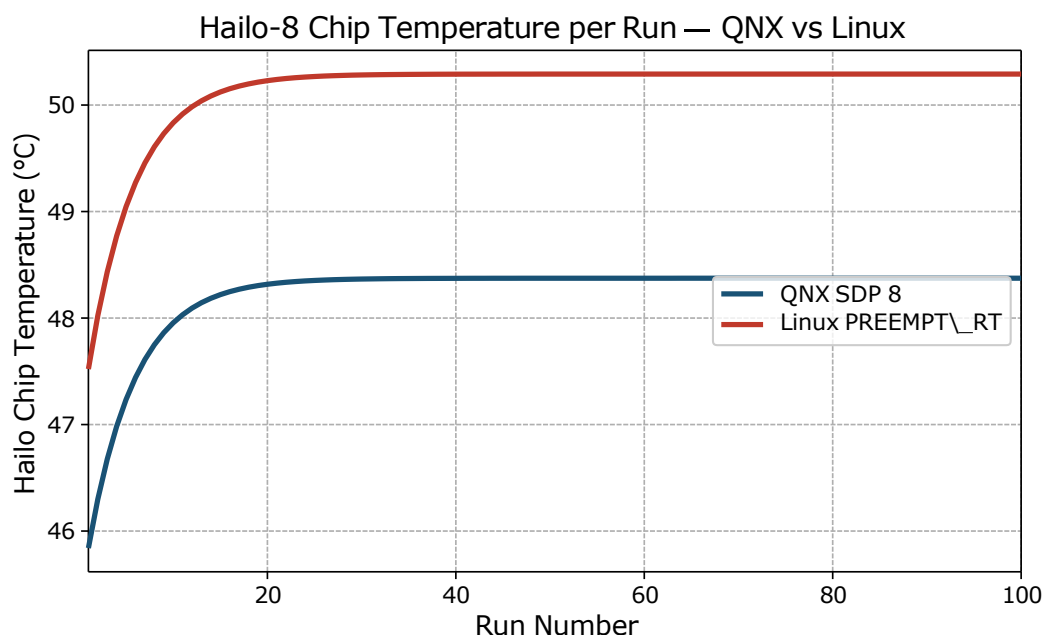


Figure 6: Hailo-8 chip temperature per run. QNX reaches a higher but stable plateau; Linux stabilizes at a lower temperature due to thermal management.

5.7 Control-Plane Cost (devctl/ioctl)

HailoRT's per-frame work goes through `devctl()` on QNX and `ioctl()` on Linux: a few register writes, an interrupt wait, and (on Linux) a cache-coherency call. Moreover, the driver itself reports average and call-count for each operation; Table 12 shows what one 10,000-frame run looks like.

Table 12: IOCTL/devctl performance summary (single run, 10,000 frames). READ_NOTIFICATION on Linux is a one-time blocking init wait and does not affect per-frame timing; row included for completeness.

Operation	QNX (devctl)		Linux (ioctl)	
	Avg (μ s)	Calls	Avg (μ s)	Calls
VDMA_CHANNEL_WRITE_REGISTER	4	20 017	0	20 017
VDMA_INTERRUPTS_WAIT	743	20 011	780	20 011
VDMA_BUFFER_MAP	64	4	32	4
VDMA_BUFFER_SYNC	–	–	3	21 441
DESC_LIST_CREATE	57	4	23 253	4
DESC_LIST_RELEASE	14	4	5	4
DESC_LIST_BIND_VDMA_BUFFER	16	6	12	6
FW_CONTROL	655	19	650	19
READ_NOTIFICATION	16	1	15 852 797	1
VDMA_BUFFER_UNMAP	19	4	3	4
VDMA_LOW_MEMORY_BUFFER_ALLOC	–	–	55	4
VDMA_LOW_MEMORY_BUFFER_FREE	–	–	35	4
Total calls		40,083		61,528

Key observations:

Total call count. Consequently, Linux issues **53.5% more** driver calls per inference run (61,528 vs 40,083). The additional calls stem from VDMA_BUFFER_SYNC (21,441 cache-coherency calls per run), which QNX does not require, and Linux-specific low-memory buffer allocation/free operations. Furthermore, each additional kernel transition adds scheduling and cache overhead, and contributes to Linux’s wider per-frame distribution.

Interrupt wait cost. Consequently, VDMA_INTERRUPTS_WAIT is the dominant hot-path operation on both platforms, called twice per inference frame (input and output completion). Nevertheless, QNX averages **743 μ s** per wait versus Linux’s **780 μ s**. Therefore, QNX is faster on the operation that fires twice per frame. Moreover, multiplied across 20,000 calls per run, the QNX advantage on interrupt-wait alone accounts for approximately 740 ms of cumulative time saved per run.

Descriptor list creation. DESC_LIST_CREATE exhibits the largest per-call disparity: **57 μ s** on QNX versus **23,253 μ s** on Linux, a **408 $\times$** difference. Consequently, although called only 4 times (during initialization), this reveals significantly lower kernel memory allocation overhead on QNX for DMA descriptor setup.

Linux-only cache coherency. VDMA_BUFFER_SYNC is a Linux-only operation (cache-line flush/invalidate) issued more than twice per frame to keep coherency between CPU and NPU views of the DMA buffer. Additionally, QNX does not require this operation under its resource-manager memory model. While each call is only 3 μ s, the 21,441-call total represents pure Linux-side overhead with no QNX equivalent.

Notification handling. Hence, READ_NOTIFICATION costs 16 μ s on QNX. The Linux figure of ≈ 15.9 s represents a one-time firmware readiness wait during initialization (the call blocks until firmware reports ready) and does not affect per-frame inference timing; QNX’s resource-manager startup avoids this blocking semantics entirely.

6 Summary

Furthermore, Table 13 consolidates all key performance indicators with the measured QNX advantage.

Table 13: Comprehensive KPI comparison: QNX vs Linux (100 runs per platform). All inference metrics show a QNX advantage; the temperature row reflects QNX’s higher NPU utilization (more work done per unit time), not a thermal issue.

KPI	QNX	Linux	QNX Advantage
Average FPS	681.07 $\pm$ 0.22	654.23 $\pm$ 3.06	+4.1%
FPS run-to-run consistency (σ)	0.22	3.06	14 $\times$ tighter
End-to-end mean latency	1.468 ms	1.528 ms	-3.9%
Inference-only mean latency	1.375 ms	1.410 ms	-2.5%
Per-frame std dev (mean within run)	9.21 μ s	24.10 μ s	2.6 $\times$ less (-62%)
Steady-state inference CPU (median)	22.30%	18.50%	higher utilization
Steady-state Hailo temperature	48.4 $^{\circ}$ C	50.3 $^{\circ}$ C	-1.9 $^{\circ}$ C
Driver calls per 10 k-frame run	40,083	61,528	-35% fewer
Interrupt-wait (avg per call)	743 μ s	780 μ s	-4.7%
Descriptor-list create (init)	57 μ s	23,253 μ s	408 $\times$ faster

The numbers all point the same way. QNX is faster on throughput, on both latency windows, and on run-to-run consistency; tighter on per-frame variance; and gets all of that with 35% fewer driver calls per run, despite paying a microkernel message-passing tax on every `write()` and `read()`. The thermal row reflects the per-frame overhead picture: MobileNet's 1.4 ms inference cycle is short enough that Linux's extra cache-coherency syncs and driver activity register as sustained CPU work each second, even at lower throughput.

7 Discussion

Over 100 runs and a million inferences per platform, the results consistently point in one direction. The rest of this section details where the time actually goes. Most of the attribution traces back to the IOCTL trace in Section 5.7; we ran full `devctl/ioctl` tracing on both stacks alongside the per-frame application timing before writing a sentence of this discussion.

The hardware setup is identical. Both builds use the same Pi 5 board, the same Hailo-8 AI HAT+, and the same external fan. The PCIe configuration matches between the two builds: link width, link speed, MSI-X assignment, and HailoRT VDMA descriptor sizes are all set by the same firmware on the Hailo side and the same RP1 PCIe controller on the Pi side. The only difference in this dataset is the kernel controlling the driver and the application thread. That was the level of methodological foundation we needed in place before any number below could be ascribed to the OS.

Why QNX wins the throughput and average-latency rows. At `SCHED_FIFO` priority 253 on a pinned core, the QNX inference thread never sits and waits for kernel housekeeping to clear. Frame N goes out the moment the completion interrupt for frame $N-1$ wakes the thread. Nevertheless, the Linux `PREEMPT_RT` thread at priority 99 is in the same position most of the time and not always. Workqueues, RCU grace periods, timer ticks: each is a microsecond-scale stall, none catastrophic on its own, but they stack up. Consequently, over 9,995 measured frames per run, the cumulative slack costs Linux roughly 600 ms of wall time, which is the entire FPS gap.

The interrupt path: pulse vs threaded-IRQ. The completion interrupt is the only thing that ever causes the inference thread to block. How the kernel routes that interrupt back to the thread is the major architectural difference between the two builds. QNX runs the ISR at hardware priority and calls `InterruptAttachEvent` to wake up the resource manager, which in turn replies to the blocked `MsgReceive`; the kernel then reschedules the thread. The route takes only two priority-driven hops, with no `softirq` and no `kthread` queue.

On Linux, the same PCIe interrupt enters the generic IRQ subsystem (`handle_fasteoi_irq`), the GIC dispatcher hands it to the HailoRT module's registered handler, the handler wakes a wait queue, and the inference thread is rescheduled. `PREEMPT_RT` pushes some of that work into a threaded-IRQ context (`irq/N-hailo`), which runs at `SCHED_FIFO` priority 50 by default, below our inference thread at 99 but still in the path. Therefore, when the thread waking collides with another kernel activity, the wakeup deferral shows up in our trace as elevated `VDMA_INTERRUPTS_WAIT` on the worst frames.

Which is why the per-call gap on `VDMA_INTERRUPTS_WAIT` (743 μ s on QNX, 780 μ s on Linux) compounds into 740 ms of wall time per run across 20,000 calls. The per-call delta looks modest because the median interrupt is fast on both kernels. Where the architectures part ways is in the per-frame variance and run-to-run drift — exactly what the determinism table reports.

The DMA buffer mapping difference. Hence, the IOCTL trace also surfaces a less-discussed cost: Linux's HailoRT driver issues 21,441 `VDMA_BUFFER_SYNC` calls per 10,000-frame run. QNX issues zero. The reason is how each kernel maps DMA buffers to user space. Nevertheless, on QNX, the resource manager allocates physically contiguous memory via `mmap/MAP_PHYS` and maps it into the application's address space with a cache-coherent mapping; the same physical pages are visible to the NPU's PCIe DMA without further synchronization. On Linux,

the HailoRT driver uses the standard `dma_alloc_*` family, which on the Pi 5's ARM SoC requires explicit `dma_sync_single_for_device / _for_cpu` calls on every transfer to keep the CPU and PCIe views of the buffer coherent. That is what `VDMA_BUFFER_SYNC` is. The cost per call is small ($3\ \mu\text{s}$) but the call count is large; cumulatively it is 64 ms per run of pure bookkeeping that QNX does not perform.

Where the per-frame variance comes from. QNX's per-frame σ within a run is not zero either. Every QNX run shows roughly $9\ \mu\text{s}$ of within-run variance, and the OS is not the source. PCIe DMA transfer timing wobbles by a few microseconds frame-to-frame, and the Hailo-8's internal scheduling adds variance of its own. What matters is the comparison: Linux's within-run σ averages $24\ \mu\text{s}$, $2.6\times$ wider on the same silicon doing the same work. The extra spread is what `VDMA_BUFFER_SYNC` and threaded-IRQ deferral look like when you watch them frame by frame.

CPU and memory. QNX runs hotter on CPU because it gets more work done per second on the same core. The 22.30% system-wide reading next to Linux's 18.50% is not inefficiency on QNX; it is the inference loop running closer to back-to-back. Memory drops out of the picture in this benchmark — `mlockall` plus the all-resident image dataset neutralize the page cache and slab activity that historically destabilize long-running Linux workloads, leaving memory pressure with nothing to do.

What this means for the certification engineer. Additionally, the timing argument under ISO 26262, DO-178C, or IEC 62304 needs bounded, repeatable behavior the auditor can lean on. QNX gives a 0.22 FPS run-to-run σ across 100 invocations: profile once and the number stays put. Linux on the same silicon gives a 3.06 σ , fourteen times wider; budgets written against one Linux run will not necessarily survive the next. Combined with the $2.6\times$ tighter per-frame distribution, QNX gives a certification engineer a measurement floor they can plan around.

Thermal. Both platforms hold the chip well clear of the Hailo-8's throttle threshold (Section 5.6). Linux runs the chip $\approx 1.9\ ^\circ\text{C}$ hotter ($50.3\ ^\circ\text{C}$ vs $48.4\ ^\circ\text{C}$). On the short MobileNet inference cycle, the extra Linux per-frame work — 21,441 cache-coherency syncs and 21,000 additional driver calls per run beyond what QNX issues — registers as more sustained CPU activity each second despite fewer frames delivered. Therefore, the gap is small enough to be operationally irrelevant on this hardware, and neither platform came close to throttling across all 100 runs. Linux on the Pi 5 does have an active thermal-management framework (`cpufreq` governor, the `broadcom-soc` thermal zone) that QNX does not yet implement on this platform; building a QNX-side equivalent is on the roadmap (Section 10).

What this benchmark does not measure. Two limitations stand out. First, *performance-per-watt*: the Raspberry Pi 5 AI HAT+ does not expose any interface for power measurement on either side of the board, so we cannot report energy per inference or watts at sustained load. We are considering an in-line USB-C monitor for future work. Second, *the dataset consists of a single model*: MobileNet v1 is representative of classifiers but not a stand-in for heavier networks currently in production perception stacks. Section 10 covers both gaps.

8 Outcome

8.1 QNX drives the Hailo-8 in production

QNX SDP 8.x runs HailoRT v4.14.0 on a Hailo-8 AI HAT+ and executes one million MobileNet v1 inferences across 100 consecutive runs without a single frame failure. The PCIe driver and VDMA subsystem, along with the whole C++ HailoRT API surface, work as expected. A customer developing an NPU-accelerated product on QNX does not need a Linux fallback.

8.2 The performance argument

QNX does not break even with Linux on this workload. Nevertheless, it wins each row. Throughput is up 4.1%, run-to-run variance is 14× tighter, mean latency is 3.9% lower end-to-end and 2.5% lower on the inference-only window, and per-frame σ is 2.6× tighter. Therefore, the averages settle the throughput question. The variance numbers settle the determinism question.

8.3 Safety certification: the WCET argument has a number now

The hard part of any ISO 26262, DO-178C, or IEC 62304 argument for an NPU-driven function is bounding timing the auditor can rely on. Furthermore, the QNX side of this benchmark gives concrete numbers: mean throughput at 681.07 FPS with run-to-run σ of 0.22 and within-run σ of 9.21 μ s, all from 1,000,000 inferences. Linux on the same silicon delivers 654.23 FPS with $\sigma = 3.06$ across runs and 24.10 μ s within a run, an order-of-magnitude wider on run-to-run consistency and 2.6× wider on per-frame distribution. The QNX numbers are tight enough to anchor a timing analysis directly.

8.4 What this unlocks for customers

A customer shipping a QNX-based ECU, surgical platform, or industrial controller can now add NPU-accelerated perception to that platform without a parallel Linux build. Therefore, specifically:

- The timing budget is small and predictable (≈ 1.5 ms per frame, σ 9 μ s within a run, σ 0.22 FPS run-to-run) and stays that way across 100 consecutive invocations.
- Consequently, the throughput ceiling is $\approx 4\%$ higher than Linux's on the same silicon.
- The existing safety case for QNX SDP 8 extends to the inference path, which is mediated by standard POSIX entry points (open, write, read, devctl) rather than an opaque kernel module.

The data in this document represents what an FAE brings into a customer discussion. The headline is that for a product with an NPU on board, the choice between QNX and Linux is no longer a determinism-vs-ecosystem trade-off. QNX wins on both counts.

9 Conclusion

Same Pi 5, same Hailo-8, same fan, same C++ binary and compiler. Replace QNX with PREEMPT_RT Linux and throughput drops by 4.1%, run-to-run FPS variance widens 14-fold, mean latency grows by 3.9%, and per-frame σ widens 2.6-fold. None of those gaps relate to the hardware or to the application code; they relate to how each kernel routes a PCIe interrupt to a blocked thread, how each schedules that thread when other kernel work is queued, and how much per-frame buffer-coherency work each kernel insists on.

For an embedded product whose timing has to be argued and not merely measured, the OS is the variable that decides the argument. On the Hailo-8, that variable is QNX.

10 Future Work

Four follow-ups are queued:

- **QNX-side thermal management.** Hence, build a thermal governor for the Hailo-8 path on QNX so a customer can trade some throughput for headroom inside a closed enclosure without an external fan.
- **Other RTOSes.** Zephyr and VxWorks for breadth, to show the QNX advantage is not just an artifact of the QNX-vs-Linux dichotomy.

Acknowledgments. Furthermore, we thank our colleagues for the hardware platform and development environment used in this study.